

Jane Street Puzzle July 2026

Andrew Ely

					37		1100
			23		138		
528							
	449			16			
	750		88		272	1	
0							

The board above has been tiled with the 12 pentominoes (plus a 2-by-2 tetromino) into 13 regions. Think of each of these 13 regions as constructed out of 1-by-1-by-1 cubes. We need to add a **tower** to each region. A tower is an additional size-1 cube placed on one of a region's squares.

After adding these towers, place a knight at the **bottom-left square**. It then proceeds to make knight's moves until it has visited all the towers. It never visits the same space twice. (A *move* on this board involves travelling 0 units in one dimension, 1 in another, and 2 in the third. The knight is allowed to "pass through" towers as it moves.)

But there's a catch: As you can see, the knight starts with a score of 0. On its N th move, its score increases by N if the move is to a location at the same altitude as the square it moved from. If, instead, it moves up, the score is multiplied by N . And finally, if it moves down, the score is divided by N . This last type of move is only allowed if the score is evenly divisible by N .

Every three moves, up until move #18, the knight wrote down its score upon arriving at a given square. From then on it only wrote down its score every K moves, for some larger value K . Using this information, **can you reconstruct the knight's path?**

After filling all the remaining visited squares with the missing score values, find the *unvisited* squares. For each of these squares, compute the sum of the scores in any orthogonally adjacent squares that were part of the knight's path. The answer to this puzzle is the sum of these "neighbor sums" from the unvisited squares.

1 Understanding the Problem

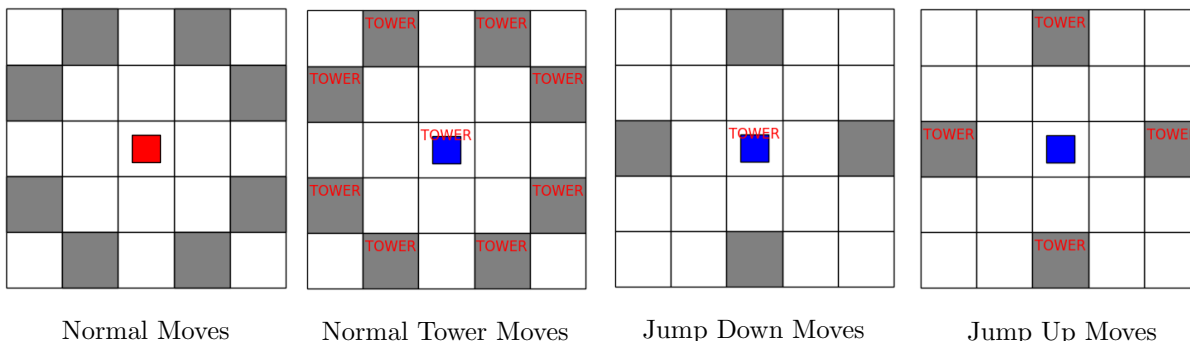
Before attempting to develop a solution to the problem, it is important to understand what the problem at hand is. We are presented with 5 paragraphs in the instructions, and each paragraph provides information about the puzzle. We can break down the information in each paragraph as follows.

1.1 The boards construction

The 12 pentominos refers to the 12 distinct shapes constructed by arranging 1 by 1 equal sized squares without rotation and reflection, and the 2 by 2 tetromino refers to the center square region (where the number 16 is). The instructions mention adding 1 tower to each region, and these towers are size-1 cubes. Notice that there can be at most one tower on a square, so there is only two levels of altitude. A normal 2D chess board is 8 by 8 squares, resulting in 64 discrete spaces for a piece to move to. Adding the towers seems to add an additional dimension, consisting of two levels. However, our board does not really become a 8x8x2 discrete metric space because any assignment of towers to the board results in exactly 64 possible spaces the knight can move. For this reason, it can be helpful to think of our board as simply a 8x8 2D board with the possible attribute of a tower existing in any given cell, which affects the knight's movement as follows.

1.2 How the knight moves

In standard chess, the knight moves two squares in one direction, and one square in the other direction like the shape of an "L". If we consider a normal chess board with dimensions left/right and up/down, a knight has 2 options of which dimension to move 2 spaces in. Then, we chose which direction for each dimension to move, resulting in the 8 possible moves for a knight to make, not considering moves that are out of the boundaries of the board. Following this same construction, by adding a third dimension to the movement, we have 3 choices for which dimension to move 2 spaces in, and for each of the 3 choices we have 2 choices for which dimension to move 1 space in, while the final dimension follows with 0 spaces. However, we must realize that we cannot move 2 spaces in the up / down dimension because we are only to place one tower on each region, there is no possible way to move 2 units up or down. Then we result with only 4 possible ways to decide what dimensions we will move 2,1, and 0 spaces in. For each of these 4 ways, we have 2 choices for if we should move positive or negative for each of the dimensions we assigned 2 and 1 space movement to. This results in 16 possible movements a knight can make at any given time, without considering towers/numbers/boundaries. Also notice that the only way for the knight to make any type of move is as follows: normal chess moves must be from either no tower square to no tower square or tower square to tower square, jumping down moves must be from tower square to non tower square, and jumping up moves must be from no tower square to tower square.



1.3 How the score accumulates

The knight begins at the bottom left square marked with the number 0, awaiting to make the first (N=1) move. If at any move the knight makes a normal move as depicted above, the score accumulates by N. If the knight jumps up to a tower, the score is multiplied by N, and if the knight jumps down the score is divided by N which is only allowed if the resulting division is an integer. This division rule restricts how often this move is available.

1.4 The marked numbers

The numbers represent where the knight marked down its score. There are 12 squares with numbers, and for the first 18 moves the knight marked down every 3 moves. We also start on move 0 on the numbered square 0. This means 7 of the 12 numbered squares will be explored after the first 18 moves. This is critical because after the first 18 moves, the space between when the knight records the number changes.

1.5 Constructing the final sum

After recovering the knights path, computing the final sum is straightforward.

2 Solving the problem

After fully understanding the problem, we can begin brainstorming an approach for solving. One realization is that for any given location, the knight has at most 8 possible spots it can move to. With every 3 moves requiring the knight to be on a numbered cell for the first 18 moves, the search space for exploring possible paths may actually be quite small. As it turns out, simply evaluating where the knight can move to at each move and ruling out impossible paths is a feasible approach.

We perform a depth-first search over partial knight paths. At each step, it generates every move consistent with the knight's geometry, the current tower height, and the arithmetic rule associated with the move. It then prunes any path that revisits a cell, places multiple towers in one region, or conflicts with a required numbered cell. The search is divided after move 18 because the spacing of the numbered clues changes at that point. A path is accepted once it visits every numbered cell and places exactly one tower in every region. The psuedo code below provides the alogrithm used to solve the problem.

2.1 Checking possible moves

Algorithm 1 Generate possible moves

```
function GETPOSSIBLEMOVES(currentCell,currentMoves)
    currentScore  $\leftarrow$  currentCell.num
    currentMoveNumber  $\leftarrow$  length(currentMoves)
    possibleMoves  $\leftarrow$  [ ]
    moveDirections  $\leftarrow$  [
        (2, 1, 0), (-2, 1, 0), (2, -1, 0), (-2, -1, 0),
        (1, 2, 0), (-1, 2, 0), (1, -2, 0), (-1, -2, 0),
        (0, 2, 1), (0, -2, 1), (2, 0, 1), (-2, 0, 1),
        (0, 2, -1), (0, -2, -1), (2, 0, -1), (-2, 0, -1) ]

    for move in moveDirections do
        nextRow  $\leftarrow$  currentCell.row + move[0]
        nextCol  $\leftarrow$  currentCell.col + move[1]
        heightChange  $\leftarrow$  move[2]
        if nextRow < 0 or nextRow > 7 or nextCol < 0 or nextCol > 7 then
            continue
        end if
        nextCell  $\leftarrow$  board[nextRow][nextCol]
        if nextCell.tower  $\neq$  None then
            continue
        end if

        if heightChange == 0 then ▷ Normal knight move
            newScore  $\leftarrow$  currentScore + currentMoveNumber
            newTowerStatus  $\leftarrow$  currentCell.tower
        else if currentCell.tower == True and heightChange == -1 then ▷ Move downward from a
tower
            if currentScore mod currentMoveNumber  $\neq$  0 then
                continue
            end if
            newScore  $\leftarrow$  currentScore/currentMoveNumber
            newTowerStatus  $\leftarrow$  False
        else if currentCell.tower == False and heightChange == 1 then ▷ Move upward onto a tower
            newScore  $\leftarrow$  currentScore · currentMoveNumber
            newTowerStatus  $\leftarrow$  True
        else
            continue
        end if

        if nextCell.num == None or nextCell.num == newScore then
            newMove.row  $\leftarrow$  nextRow
            newMove.col  $\leftarrow$  nextCol
            newMove.num  $\leftarrow$  newScore
            newMove.tower  $\leftarrow$  newTowerStatus
            newMove.region  $\leftarrow$  nextCell.region
            possibleMoves.append(newMove)
        end if
    end for
    return possibleMoves
end function
```

2.2 Constructing the moves

Algorithm 2 Construct a sequence of moves

```

trackedMoves ← [[{'row': 7, 'col': 0, 'num': 0, 'tower': True, 'region': 't'}],
                  [{'row': 7, 'col': 0, 'num': 0, 'tower': False, 'region': 't'}],]
first18moves ← []
function GETFIRST18MOVES
    while trackedMoves do
        currentMoves ← trackedMoves.pop()
        currentCell ← currentMoves.LastElement()
        possibleMoves ← getPossibleMoves(currentCell, currentMoves)
        for p in possibleMoves do
            if CheckMove(p, currentMoves) then
                newMoves ← currentMoves.copy()
                newMoves.append(p)
                trackedMoves.append(newMoves)
            end if
        end for
        if length(currentMoves) == 19 then
            first18moves.append(currentMoves)
        end if
    end while
end function

function GETRESTOFMOVES(first18moves)
    count ← 0
    for k in range(4,10) do
        trackedMoves ← first18Moves.copy()
        while trackedMoves do
            currentMoves ← trackedMoves.pop()
            currentCell ← currentMoves.LastElement()
            possibleMoves ← getPossibleMoves(currentCell, currentMoves)
            for p in possibleMoves do
                if CheckMove(p, currentMoves) then
                    newMoves ← currentMoves.copy()
                    newMoves.append(p)
                    trackedMoves.append(newMoves)
                    if CheckFinished(newMoves) then
                        return newMoves
                    end if
                end if
            end for
            if length(currentMoves) == 19 then
                first18moves.append(currentMoves)
            end if
        end while
    end for
end function

```

▷ We reached a sequence of 18 moves.

▷ ****WE CANT HAVE K ; 9****

▷ We reached a sequence of 18 moves.